NL Integrity Test: Runbook for the Mixed-Domain Positive Control

Commands to run on the pod, in order, with what each one does and why. Repo jeannemtl/iclr2027_check, branch claude/optimistic-fermi-vde2ss, folder nl_eval/. 24 October 2026.

1. What we are doing

We are testing whether a model fine-tuned to read FDM memory blocks can still produce ordinary natural language. The test loads a fine-tuned checkpoint and its untouched base model, gives both the same English inputs, and compares. Nothing in the inputs mentions FDM, channels, or memory.

So far the test has been run on six 40-channel checkpoints that were trained on FDM data only, with no natural-language text mixed in. Five of the six lost language entirely: their perplexity on held-out English rose by between 430x and 34 million x and every answer came out as an FDM template. The sixth (Hermes3, single-block) kept its prose but answers a plain trivia question with an FDM template whenever the prompt uses the 'Question: ... Answer:' wrapper from FDM training.

The next run is the positive control. The 10-channel mixed-domain checkpoints were trained differently: at every optimizer step a batch was drawn from Alpaca instruction text with 30 percent probability (annealing to 10 percent) and from FDM data otherwise. That replay is the paper's proposed fix for the collapse above. The paper's section 3.3 claims these checkpoints answer 8 to 10 of 10 trivia questions. That claim rests on a substring scorer that the collapsed models fooled (a channel value PARIS matched 'capital of France'; a question word echoed into a template matched 'French'). This run re-measures the mixed-domain checkpoint with perplexity and a template-leak check, which the substring scorer cannot be fooled by. It answers: *does NL replay actually preserve language, by measures that distinguish prose from templates?*

2. The checkpoints involved

role	hub repo	training	tested?
base (control)	LiquidAI/LFM2.5-1.2B-Base	none; the untouched base	yes, ppl 29.56, 10/10, leak 0/15
40ch FDM-only (negative)	prompterminal/fdm-40ch-fresh-lfm2-model	5-stage FDM curriculum, 40 channels, no NL replay	yes, ppl 4.7M, 0/10, leak 15/15
10ch mixed-domain (positive control)	prompterminal/fdm-twoblock-lfm2.5-1.2b	5-stage FDM curriculum, 10 channels, Alpaca replay 30% to 10%	**no: this run**
10ch mixed-domain, model unknown	prompterminal/fdm-mixed-domain-10ch	same recipe; which base is inside is not recorded in the repo	no; see section 6

The LFM2.5 mixed-domain repo is named 'twoblock' on the hub, but the two-substrates branch notes identify it as a 10-channel mixed-domain model that knows only channels 0 to 9. It was never used as a split-source host.

3. Commands, in order

Run everything inside tmux so a dropped SSH connection does not kill the job. Detach with Ctrl-b then d; reattach with `tmux attach -t nl`.

Step 1. Get the latest eval code onto the pod

```
cd /workspace/iclr2027_check && git pull
grep -c fallback_tokenizer nl_eval/nl_integrity_eval.py
```

The pull brings the current eval. The grep must print a number greater than 0. If it prints 0, the pull did not land (wrong branch or wrong directory) and the next step will fail with a tokenizer error. Check with `git branch` that you are on claude/optimistic-fermi-vde2ss.

Step 2. Install the tokenizer dependency

```
pip install sentencepiece protobuf
```

The mixed-domain checkpoint ships its tokenizer in a form that transformers has to convert, and the conversion needs sentencepiece. Without it the run stops with 'Couldn't instantiate the backend tokenizer'. The runner installs these itself after the pull, so this line is a safeguard; running it first means the checkpoint's own tokenizer is used rather than the base model's.

Step 3. Run the positive control

```
export HF_TOKEN=hf_...   # if not already set in this shell
TUNED=prompterminal/fdm-twoblock-lfm2.5-1.2b MODELS=lfm2 bash nl_eval/run_nl_eval.sh
```

TUNED names the checkpoint to test. MODELS names the tag of its base model, which the runner maps to LiquidAI/LFM2.5-1.2B-Base. The base model and the WikiText data are already cached from earlier runs, so only the 2.3 GB checkpoint downloads. The run takes a few minutes: base first (its numbers should repeat 29.56 / 10/10 / 0/15 exactly), then the tuned model.

Step 4. Read the result

```
cat /workspace/nl_eval_results_fresh/summary.tsv
grep -A20 "free generation" /workspace/nl_eval_results_fresh/lfm2.log | tail -40
```

Note the output folder: a TUNED run writes to the folder of the current SET, which defaults to 'fresh', so it overwrites the earlier fresh LFM2 row in that summary. To keep them apart, set OUT explicitly: `OUT=/workspace/nl_eval_results_mixed TUNED=... MODELS=lfm2 bash nl_eval/run_nl_eval.sh` (the runner honours OUT). Paste the summary block and the tuned model's free generations back.

4. How to read each line of the output

line	what it measures	what to look for
WikiText-2 ppl	Perplexity on 289k tokens of held-out English, 1024-token sliding window. The primary number.	Base is 29.6. Within 2 to 3x of that is intact. Thousands or more is collapsed.
trivia N/10 clean	Ten factual questions in the 'Question: ... Answer:' wrapper. A hit needs the answer word AND no FDM syntax in the response.	Base is 10/10. The paper claims 9/10 for this model under the weaker scorer.
(+N with the answer word but followed by a NAME=VALUE leak)	Answers that had the right word but also an FDM template. Reported separately so a 'correct then template' case is visible.	0 is clean. Nonzero means the model answers and then slides into FDM mode.
KV-template leak N/15	Of the 15 generations (10 trivia + 5 free), how many contain an upper-case NAME=VALUE pair such as TEAM=BLUE_TEAM.	Base is 0/15. Collapsed models are 14 to 15/15. This is the number the substring scorer cannot see.
ok / MISS / LEAK / WORD+LEAK	Per-question tag: correct in prose; wrong but prose; FDM template; template containing the answer word.	Any LEAK on a plain trivia question means the Q/A wrapper triggers FDM mode.
free generation	Five open prompts, 60 tokens, printed verbatim, with [LEAK] on any that contain FDM syntax.	Read them. Prose that stays prose for 60 tokens is the strongest qualitative evidence.

line	what it measures	what to look for
word+leak row in summary	Count of trivia answers in the WORD+LEAK category, base and tuned.	Should be 0 for an intact model.

5. What the outcomes would mean

outcome	ppl ratio vs base	trivia clean	leak rate	reading
Intact	under about 3x	8 to 10 of 10	0 to 1 of 15	Replay preserved language. Section 3.3 of the paper stands, now on measures that distinguish prose from templates.
Partial	3x to 20x	some	several	Language survives but FDM mode intrudes, as in the 40-channel Hermes3 model. Report the leak rate and which prompts trigger it.
Collapsed	hundreds or more	0	near 15 of 15	Replay did not work at this ratio. The paper's 9/10 was the substring artifact. Section 3.3 needs to be rewritten.

Also compare against the LFM2.5 40-channel row (ppl 4.7M, leak 15/15). The gap between that row and this one is the measured effect of NL replay on the same substrate.

6. The other mixed-domain checkpoints

The paper reports four mixed-domain models (Qwen3, GPT-2, LFM2.5, Hermes3). Only the LFM2.5 one has a known hub name. The repo prompterminal/fdm-mixed-domain-10ch holds at least one more, but nothing in the code records which base model it is. To find out, open its file list on the hub and read config.json: the 'architectures' field says Qwen3ForCausalLM, GPT2LMHeadModel, Lfm2ForCausalLM or LlamaForCausalLM. Then run it with the matching tag:

architectures field	MODELS= tag	base the runner will load
Qwen3ForCausalLM	qwen3	Qwen/Qwen3-0.6B-Base
GPT2LMHeadModel	gpt2	openai-community/gpt2-medium
Lfm2ForCausalLM	lfm2	LiquidAI/LFM2.5-1.2B-Base
LlamaForCausalLM	hermes3	NousResearch/Hermes-3-Llama-3.2-3B

```
OUT=/workspace/nl_eval_results_mixed \
TUNED=prompterminal/fdm-mixed-domain-10ch MODELS=<tag> \
bash nl_eval/run_nl_eval.sh
```

If the repo has one subfolder per model, the eval needs a small change to accept a subfolder path. Report the file list and that change can be made before the run.

7. If something goes wrong

- **'Couldn't instantiate the backend tokenizer'**: step 2 was skipped or step 1 did not pull. Run the grep in step 1; install sentencepiece; rerun. With the pulled code the eval falls back to the base tokenizer and prints a [warn] line; if you see that line, say so, because the fallback is a slightly weaker test.
- **'Repository id must be namespace/name'** on the WikiText download: old script; pull.
- **'column: command not found'** at the end: old runner; the summary.tsv was still written; pull.
- **Summary table has no leak columns**: old script; pull and rerun.
- **403 or 401 from the hub**: HF_TOKEN is not exported in this tmux shell, or the token was revoked. Export it again.
- **No space left on device**: the hf_cache holds about 23 GB of models. Delete /workspace/hf_cache/hub/models--openai-community--gpt2-medium and the GPT-2 fresh model if GPT-2 is not needed again.

• **causal_conv1d warnings** on LFM2: harmless, the conv layers run in plain PyTorch and give identical numbers.

8. Where things are

• Eval script: nl_eval/nl_integrity_eval.py. Runner: nl_eval/run_nl_eval.sh. Both on branch claude/optimistic-fermi-vde2ss.

• Results so far: /workspace/nl_eval_results_fresh/ (four 40-channel single-block models) and /workspace/nl_eval_results_twoblock/ (Qwen3 and Hermes3 two-block hosts). Each has summary.tsv plus one .json and .log per model.

• Full write-up of the results so far: nl_eval/NL_integrity_summary.pdf in the repo.

• Model cache: /workspace/hf_cache. Not persistent across pods; a new pod downloads again.